

Detecção de Viaturas em Vídeos Digitais com YOLOv8

Luís A.V. Ferreira

Versão
5 de Maio de 2025

Resumo

Este trabalho apresenta um algoritmo para detecção de viaturas em vídeos digitais, utilizando o modelo YOLOv8 integrado com a biblioteca OpenCV. O objetivo é identificar e visualizar viaturas em cenários rodoviários, desenhando caixas delimitadoras (Caixa Delimitadora) em tempo real, com aplicações em gestão de tráfego, segurança rodoviária e condução autónoma. A abordagem combina eficiência computacional, precisão otimizada e pós-processamento avançado, incluindo variantes de supressão de não-máximos (NMS). Os resultados preliminares indicam alta precisão em condições favoráveis (mAP@0.5 de 0,85 para automóveis), embora limitações como oclusões e iluminação noturna persistam. Extensões potenciais incluem rastreio com DeepSORT e estimativa de velocidade.

Conteúdo

Conteúdo	1
1 Introdução	2
2 Fundamentação Teórica	3
2.1 Detecção de Objectos: Formalização Matemática	3
2.2 Evolução dos Algoritmos Matemáticos no YOLO	4
2.3 Métricas de Avaliação	5
2.4 Arquitectura do YOLOv8	6
2.5 Arquitecturas CNN	6
2.6 Design Consciente do Hardware	6
2.7 Comparação com Outros Detectores	7
2.8 Caixas Âncora vs. <i>Anchor-Free</i>	7
2.9 Pós-processamento: NMS e Variantes	7

3	Trabalhos Relacionados	8
4	Materiais e Métodos	8
4.1	Ambiente Experimental	8
4.2	Justificação das Escolhas	9
4.3	Fluxo de Inferência	9
4.4	Configuração do Modelo	9
4.5	Avaliação Quantitativa	11
4.6	Limitações Técnicas	11
5	Resultados e Discussão	11
6	Conclusão	11
7	Trabalhos Futuros	12

1 Introdução

A detecção de viaturas em vídeos digitais captados em vias públicas é um pilar fundamental para sistemas de visão computacional aplicados à gestão de tráfego, segurança rodoviária e condução autónoma. Como garantir detecção precisa em tempo real com hardware limitado? Como mitigar o impacto de condições adversas, como oclusões ou baixa iluminação? Estas questões orientam o presente estudo, que propõe um algoritmo baseado no modelo YOLOv8 (*You Only Look Once*) integrado com a biblioteca OpenCV para processar vídeos, identificar viaturas e desenhar caixas delimitadoras em tempo real.

O YOLO, inicialmente proposto por Redmon et al. [1], revolucionou a detecção de objectos ao processar imagens numa única passagem através de uma rede neuronal convolucional (CNN), alcançando velocidades de 45 fps com 63,4% de mAP no conjunto VOC 2007+2012 [2]. Evoluções como YOLOv3, YOLOv6, YOLOv8 e YOLO11 [3, 4, 5, 6] introduziram melhorias significativas, desde o uso do framework Darknet até arquitecturas modernas como EfficientRep, que otimizam a eficiência computacional através de designs conscientes do hardware [7]. Este trabalho foca-se na integração otimizada do YOLOv8 com OpenCV, detalhando o fluxo de processamento, desde a leitura de vídeos até à visualização, com extensões para rastreio e estimação de velocidade [8, 9]. Desafios como a necessidade de dados anotados [10] e limitações de hardware [11] são considerados, especialmente em ambientes urbanos dinâmicos.

Além das aplicações em segurança rodoviária e condução autónoma, a detecção de viaturas desempenha um papel crucial em sistemas de transporte inteligentes (ITS), como a optimização semafórica dinâmica, o controlo de acessos em zonas urbanas, a vigilância urbana e a monitorização da poluição gerada pelo tráfego. Estas aplicações sublinham a relevância prática do

presente estudo, que visa contribuir para o avanço de soluções robustas e escaláveis em visão computacional.

2 Fundamentação Teórica

2.1 Detecção de Objectos: Formalização Matemática

A detecção de objectos é formalizada como um problema de aprendizagem supervisionada, onde um modelo $\hat{y} = f(x; \theta)$ mapeia uma imagem de entrada $x \in \mathbb{R}^{H \times W \times C}$ (altura H , largura W , canais C) para uma previsão $\hat{y}$, com parâmetros θ . A saída $\hat{y}$ inclui um conjunto de caixas delimitadoras $B = \{(x_1, y_1, x_2, y_2, c, s)\}$, onde (x_1, y_1, x_2, y_2) são as coordenadas da caixa, c é a classe (e.g., automóvel, camião) e s é a pontuação de confiança. A função de perda combina três componentes principais [1]:

$$\mathcal{L} = \lambda_{\text{loc}} \mathcal{L}_{\text{loc}} + \lambda_{\text{conf}} \mathcal{L}_{\text{conf}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}},$$

onde:

- $\mathcal{L}_{\text{loc}}$: Perda de localização, que penaliza erros nas coordenadas das caixas delimitadoras. No YOLOv1, usa erro quadrático médio (MSE) para o centro (x_i, y_i) e raízes quadradas para largura e altura $(\sqrt{w_i}, \sqrt{h_i})$ para penalizar mais severamente erros em caixas menores, conforme:

$$\mathcal{L}_{\text{loc}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right].$$

O YOLOv8 adota métricas avançadas, como CIoU, e *Distribution Focal Loss* (DFL), que modela coordenadas como distribuições para maior precisão [12, 13].

- $\mathcal{L}_{\text{conf}}$: Perda de confiança, que avalia a probabilidade de um objecto estar presente. No YOLOv1, penaliza a diferença entre o *confidence score* previsto ($\hat{C}_i$) e o verdadeiro ($C_i = 1$ se há objecto, 0 caso contrário), com ponderação reduzida ($\lambda_{\text{noobj}} = 0,5$) para células sem objectos, evitando dominância do gradiente [1]. O YOLOv8 usa entropia cruzada binária (BCE) com logits para maior estabilidade [5].
- $\mathcal{L}_{\text{cls}}$: Perda de classificação, que penaliza erros nas probabilidades de classe. No YOLOv1, usa MSE, assumindo $p_i(c) = 1$ para a classe correcta e 0 para outras, enquanto o YOLOv2 introduz previsões por caixa âncora, e o YOLOv8 adota BCE para alinhar-se com distribuições probabilísticas [5, 14].

Os pesos λ equilibram as contribuições de cada termo, ajustados durante o treino para otimizar o desempenho. No YOLOv1, $\lambda_{\text{coord}} = 5$ prioriza a localização, enquanto $\lambda_{\text{noobj}} = 0,5$ reduz a influência de células sem objectos [1]. A evolução para o YOLOv8, com uma abordagem *anchor-free*, elimina a dependência de caixas âncora, simplificando a configuração e

melhorando a generalização para objectos de tamanhos variados [5].

Extensões futuras do DFL podem incorporar incerteza nas previsões, modelando caixas delimitadoras como distribuições com matrizes de covariância, inspiradas em optimizações de conjuntos semi-definidos positivos [15].

2.2 Evolução dos Algoritmos Matemáticos no YOLO

O YOLO (*You Only Look Once*) evoluiu significativamente desde a sua introdução em 2016, incorporando algoritmos matemáticos que melhoram a velocidade, precisão e robustez na detecção de objectos. Esta subsecção sumariza as principais técnicas usadas nas versões do YOLO, com foco na sua relevância para a detecção de viaturas com YOLOv8 [1, 5].

- **YOLOv1 (2016)**: Processa a imagem numa única passagem através de uma rede neuronal convolucional (CNN), dividindo-a numa grelha $S \times S$. Cada célula prevê B caixas delimitadoras e probabilidades de classe. A perda de localização usa erro quadrático médio (MSE) para centros (x_i, y_i) e raízes quadradas para dimensões $(\sqrt{w_i}, \sqrt{h_i})$, penalizando erros em caixas menores [1].
- **YOLOv2 (2017)**: Introduce caixas âncora geradas por *k-means++* com distância baseada em IoU, optimizando a inicialização. A normalização em lote (*batch normalization*) estabiliza o treino, reduzindo a covariância interna dos gradientes [14, 16].
- **YOLOv3 (2018)**: Usa a espinha dorsal Darknet-53 com conexões residuais e detecção multi-escala via *Feature Pyramid Networks* (FPN). Substitui o MSE por entropia cruzada binária (BCE) com logits para classificação multi-rótulo. A supressão de não-máximos (NMS) elimina caixas redundantes com base no IoU [3, 17].
- **YOLOv4 (2020)**: Adota *Mosaic Data Augmentation* para enriquecer o contexto do treino e *DropBlock* para regularização. A perda CIOU, definida como:

$$\mathcal{L}_{\text{CIOU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^g)}{c^2} + \alpha v,$$

penaliza localização com base na distância euclidiana (ρ), diagonal do rectângulo envolvente (c), e alinhamento angular (v) [18, 19].

- **YOLOv5 (2020)**: Implementado em PyTorch, usa CSPNet como espinha dorsal e suporta caixas âncora auto-aprendidas. Mantém CIOU como perda padrão [19].
- **YOLOv6 (2022)**: Introduce uma arquitectura modular com *anchor-free* opcional e blocos BepC3. Usa *Varifocal Loss* (VFL) para classificação e CIOU/DFL para regressão, com *Adaptive Training Sample Selection* (ATSS) para atribuição dinâmica de rótulos [4, 20].
- **YOLOv7 (2022)**: Incorpora E-ELAN (*Efficient Layer Aggregation Network*) e cabeças auxiliares para treino profundo. Usa *Optimal Transport Assignment* (OTA) para atribuição de rótulos, baseado em transporte óptimo [21].
- **YOLOv8 (2023)**: Adota uma abordagem *anchor-free*, prevendo directamente coorde-

nadas de caixas. Usa uma cabeça desacoplada para regressão e classificação, com perda combinada de *Distribution Focal Loss* (DFL) e CIoU. O DFL modela coordenadas como distribuições, focando valores próximos dos alvos verdadeiros [5, 13]. Integra Soft-NMS para cenários densos.

Versões recentes, como YOLOv9 e YOLOv11, introduzem *Programmable Gradient Information* (PGI) e *Generalized Efficient Layer Aggregation* (GELAN) para maior precisão, mas não são usadas neste trabalho, sendo perspectivas para futuras optimizações [6]. A table 1 resume as principais técnicas matemáticas usadas no YOLO.

Tabela 1: Técnicas matemáticas nas versões do YOLO.

Técnica	Tipo	Versões
Convoluções e CNNs	Extracção de características	Todas
IoU / GIoU / CIoU	Métrica de localização	v1–v8
<i>k-means++</i> (IoU-based)	Clustering para caixas âncora	v2–v5
Darknet, CSPNet, E-ELAN	Espinhas dorsais CNN	v3–v8
FPN + PAN	Deteção multi-escala	v3–v8
NMS, Soft-NMS, DIoU-NMS	Supressão de redundâncias	Todas
<i>Mosaic Data Augmentation</i>	Aumento de dados	v4–v8
ATSS, SimOTA, OTA	Atribuição de rótulos	v6–v8
MSE, BCE, VFL, DFL	Funções de perda	Todas
<i>Anchor-free</i>	Regressão directa de caixas	v6–v8

2.3 Métricas de Avaliação

A avaliação de modelos de detecção de objectos baseia-se em métricas como a Média da Precisão Média (mAP). A $mAP@0.5$ calcula a precisão média por classe com um limiar de IoU de 0,5, reflectindo a capacidade do modelo de detectar objectos com sobreposição significativa. A $mAP@0.5:0.95$ considera limiares de IoU de 0,5 a 0,95, avaliando a robustez em diferentes níveis de precisão de localização [22]. A intersecção sobre União (IoU) mede a sobreposição entre caixas previstas e reais:

$$IoU = \frac{\text{Área de intersecção}}{\text{Área de União}}.$$

Valores de IoU próximos de 1 indicam alta precisão na localização. Além disso, curvas precisão-recall permitem analisar o equilíbrio entre deteções corretas (precisão) e a capacidade de identificar todos os objectos relevantes (recall), sendo fundamentais para otimizar limiares de confiança [22].

2.4 Arquitectura do YOLOv8

O YOLOv8 introduz avanços significativos face às versões anteriores, destacando-se pela adoção de blocos *C2f* em vez das camadas CSP tradicionais, reduzindo a complexidade computacional enquanto melhora a precisão [5]. A abordagem *anchor-free* elimina a dependência de caixas âncora predefinidas, prevendo diretamente as coordenadas das caixas delimitadoras, o que simplifica a configuração e melhora a generalização para objectos de tamanhos variados. O YOLOv8 incorpora módulos RepVGG, que utilizam convoluções 3x3 reparametrizadas para otimizar a inferência em hardware, inspirando-se em arquitecturas como o Darknet-53, mas com maior eficiência graças à reparametrização [23, 24]. A estratégia de treino inclui técnicas de aumento de dados e escalabilidade para diferentes tamanhos de modelo (e.g., YOLOv8n, YOLOv8s), tornando-o ideal para aplicações em tempo real [5].

2.5 Arquitecturas CNN

As CNNs, fundamentais para a detecção de objectos, são compostas por blocos como camadas convolucionais, funções de ativação ReLU, camadas de *pooling* (e.g., *max pooling*) e normalização em lote (*batch normalization*) [25]. No YOLOv8, a espinha dorsal CSPDarknet (*Cross Stage Partial Darknet*) otimiza a extração de características, reduzindo a complexidade computacional enquanto preserva gradientes informativos [19]. A arquitectura inclui:

- **Convoluções:** Extraem características espaciais através de filtros aprendidos.
- **Módulos C3:** Combinam convoluções padrão e residuais para melhorar a capacidade de representação.
- **SPPF (*Spatial Pyramid Pooling Fast*):** Agrega características em múltiplas escalas, robustecendo a detecção de objectos de diferentes tamanhos [5].

A cabeça de previsão gera caixas delimitadoras e probabilidades de classe, integrando características de várias camadas através de uma estrutura FPN (*Feature Pyramid Network*) + PAN (*Path Aggregation Network*) [17].

2.6 Design Consciente do Hardware

O design de redes neuronais conscientes do hardware considera a capacidade computacional e a largura de banda da memória, indo além de métricas tradicionais como FLOPs e número de parâmetros [7]. O EfficientRep, integrado no YOLOv6, exemplifica esta abordagem, utilizando blocos *RepConv* que reparametrizam ramificações 3x3, 1x1 e de identidade numa única convolução 3x3 otimizada para GPUs [23]. Para modelos maiores, o YOLOv6 introduz a unidade Bep e o bloco BepC3, que combinam a estrutura CSP com conexões lineares de *RepConv*, melhorando a eficiência em hardware de alta capacidade [4]. Esta abordagem híbrida, com arquitecturas de ramo único para modelos pequenos (YOLOv6-N/S) e multi-ramos para modelos grandes (YOLOv6-M/L), demonstra como o design consciente do hardware pode equilibrar

precisão e velocidade em diferentes cenários computacionais.

2.7 Comparação com Outros Detectores

O YOLOv8 destaca-se face a outros Detectores de objectos, como SSD, Faster R-CNN, EfficientDet e YOLOv6. O SSD [26] alcança 59 fps, mas apresenta mAP de 46,5% no COCO, inferior ao YOLOv8 (50,2% mAP a 60 fps) [6]. O Faster R-CNN [27] oferece maior precisão (mAP de 73,2% no COCO), mas é significativamente mais lento (2–3 segundos por imagem), sendo inadequado para tempo real. O EfficientDet [28] equilibra precisão e eficiência, mas consome mais memória (e.g., 52 MB vs. 6 MB do YOLOv8n) [5]. O YOLOv6, com a arquitectura EfficientRep, atinge desempenhos competitivos: YOLOv6-N (35,9% mAP a 1234 FPS), YOLOv6-S (43,5% mAP a 495 FPS), YOLOv6-M (49,5% mAP a 233 FPS) e YOLOv6-L (51,7% mAP a 149 FPS) no COCO, avaliados numa GPU NVIDIA Tesla T4 [7]. Comparado com o YOLOv6, o YOLOv8 oferece maior flexibilidade para dispositivos *edge* e uma arquitectura *anchor-free*, embora o YOLOv6-L supere ligeiramente em precisão para objectos grandes. As vantagens do YOLOv8 incluem a sua escalabilidade e optimização para cenários urbanos densos, embora seja menos preciso em objectos pequenos face a modelos de duas etapas como o Faster R-CNN.

2.8 Caixas Âncora vs. *Anchor-Free*

As abordagens baseadas em caixas âncora, como YOLOv3, utilizam caixas predefinidas para prever ajustes de posição e tamanho, o que é eficaz mas sensível à escolha das âncoras [3]. Modelos *anchor-free*, como o YOLOv8, prevêm diretamente as coordenadas das caixas, reduzindo a dependência de hiperparâmetros e melhorando a generalização [29]. As vantagens do *anchor-free* incluem:

- Menor complexidade na configuração do modelo.
- Melhor desempenho em objectos de tamanhos variados.

Porém, pode ser menos preciso em cenários com alta densidade de objectos, onde as caixas âncora fornecem maior controlo [30].

2.9 Pós-processamento: NMS e Variantes

O pós-processamento é crucial para eliminar caixas delimitadoras redundantes. A NMS tradicional seleciona a caixa com maior pontuação e elimina outras com IoU superior a um limiar (e.g., 0,5) [1]. A formulação é:

$$s_i = \begin{cases} s_i, & \text{se } \text{IoU}(A, B_i) < N_t, \\ 0, & \text{se } \text{IoU}(A, B_i) \geq N_t, \end{cases}$$

onde s_i é a pontuação da caixa B_i , A é a caixa com maior pontuação, e N_t é o limiar de IoU [31].

A Soft-NMS, proposta por Bodla et al. [32], reduz gradualmente as pontuações das caixas sobrepostas em vez de eliminá-las:

$$s_i = s_i \cdot (1 - \text{IoU}(A, B_i)), \quad \text{se } \text{IoU}(A, B_i) \geq N_t.$$

Esta abordagem melhora a detecção em cenários densos, como tráfego urbano, ao preservar caixas potencialmente corretas [33]. Outra variante, o DIOU-NMS, incorpora a distância entre centros das caixas, penalizando caixas distantes mas sobrepostas, o que aumenta a precisão em detecções de objectos pequenos [18, 34].

3 Trabalhos Relacionados

A detecção de objectos em vídeos evoluiu significativamente com as CNNs. Modelos de duas etapas, como o Faster R-CNN [27], oferecem alta precisão (mAP de 73,2% no COCO) mas são lentos (2–3 segundos por imagem), inadequados para tempo real [2]. O SSD [26] melhora a velocidade (59 FPS), mas compromete a precisão em objectos pequenos (mAP de 46,5% no COCO). O YOLO, introduzido por Redmon et al. [1] com o framework Darknet, alcançou 45 fps com 63,4% de mAP no VOC 2007+2012, utilizando uma única passagem [24, 35]. Versões como YOLOv3 [3], YOLOv6 [4] e YOLOv8 [5] adotaram arquitecturas avançadas, incluindo EfficientRep no YOLOv6, que utiliza uma abordagem híbrida (ramo único para modelos pequenos e multi-ramos para modelos grandes) para aplicações industriais [7]. O YOLOv8 atinge mAP de 50,2% no COCO a 60 fps, beneficiando de uma arquitectura *anchor-free* [6]. Detectores baseados em *Transformers*, como o DETR [30], oferecem precisão superior (mAP de 42,0% no COCO), mas requerem maior poder computacional. Trabalhos recentes, como Rodríguez-Rangel et al. [9], aplicam YOLO para estimação de velocidade, enquanto Keylabs.ai [36] explora rastreio com YOLOv8. Este trabalho destaca-se pela integração otimizada de YOLOv8 e OpenCV, com foco em ambientes urbanos e extensões para rastreio.

4 Materiais e Métodos

4.1 Ambiente Experimental

O algoritmo foi implementado em Python 3.9, utilizando PyTorch 2.0, OpenCV 4.8 e Ultralytics 8.0. Testes foram realizados num sistema com GPU NVIDIA RTX 3060 (12 GB), 16 GB de RAM e CPU Intel i7-10700. Vídeos de teste provêm dos conjuntos UA-DETRAC [37], BDD100K [38] e COCO [39], complementados por vídeos simulados em ambientes rodoviários urbanos. O modelo YOLOv8n (yolov8n.pt) foi seleccionado pela sua eficiência (15 ms por quadro em GPU) e equilíbrio entre precisão e velocidade [5].

4.2 Justificação das Escolhas

O YOLOv8 foi escolhido sobre versões anteriores (e.g., YOLOv5, YOLOv6) devido à sua arquitectura *anchor-free*, otimizações para dispositivos *edge* e suporte robusto para rastreio [5]. Comparado com frameworks como MMDetection ou Detectron2, o OpenCV foi selecionado pela sua simplicidade, eficiência em manipulação de vídeos e compatibilidade com *pipelines* em tempo real [40]. Os conjuntos de dados UA-DETRAC, BDD100K e COCO foram escolhidos pela sua diversidade e representatividade em cenários rodoviários, com anotações robustas para viaturas [37, 38, 39].

4.3 Fluxo de Inferência

O fluxo completo da inferência inclui:

1. **Leitura do Vídeo:** O vídeo é aberto com OpenCV como fluxo de quadros.
2. **Pré-processamento:** Redimensionamento para $640 \text{ px} \times 640 \text{ px}$ com *padding*, normalização para $[0, 1]$, conversão de BGR para RGB.
3. **Inferência:** Aplicação do YOLOv8n com limiar de confiança de 0,25 e NMS com IoU de 0,5.
4. **Pós-processamento:** Filtragem de caixas pequenas ($< 100 \text{ px}^2$) e aplicação de Soft-NMS para cenários densos.
5. **Visualização:** Desenho de caixas delimitadoras com rótulos e gravação do vídeo processado em formato MP4.

O pseudocódigo detalhado (algorithm 1) ilustra este fluxo, incluindo rastreio opcional com DeepSORT [36].

4.4 Configuração do Modelo

O modelo `yolov8n.pt` foi pré-treinado no conjunto COCO, que inclui 80 classes, das quais seleccionámos automóvel, motociclo, camião e autocarro (table 2). O limiar de confiança de 0,25 e IoU de 0,5 foram definidos com base em testes preliminares para equilibrar precisão e revocação [22].

Tabela 2: Correspondência entre classes internas do modelo e designações em português.

Classe (YOLO)	ID (COCO)	Designação em português
car	2	Automóvel
motorcycle	3	Motociclo
truck	7	Camião
bus	5	Autocarro

Algoritmo 1 detecção de Viaturas com YOLOv8

Entrada: Ficheiro de vídeo $V_{entrada}$, resolução $R = (w, h)$, taxa de quadros f , modelo YOLOv8 M , limiar de confiança $conf_thres$, limiar de IoU iou_thres

Saída: Vídeo processado com caixas delimitadoras, salvo como $V_{saída}$

 Abrir $V_{entrada}$ como fluxo de quadros Q

 Definir $V_{saída}$ com codec MP4V, taxa f , resolução R

 Definir classes $C \leftarrow \{\text{automóvel, motociclo, camião, autocarro}\}$

 Definir cores: Cores $\leftarrow \{\text{automóvel: (0, 255, 0), motociclo: (0, 0, 255), camião: (255, 255, 0), autocarro: (255, 0, 255)}\}$

enquanto Q tiver quadros **do**

 Ler quadro q de Q

 Redimensionar q para $640 \text{ px} \times 640 \text{ px}$, normalizar para $[0, 1]$

 Converter q de BGR para RGB

 Aplicar M a q , com $conf_thres = 0,25$, $iou_thres = 0,5$, obtendo D

para cada detecção $d \in D$ **do**

 Extrair $(x_1, y_1, x_2, y_2, \text{classe } c, \text{confiança } conf)$

se $c \in C$ e $\text{área}(d) \geq 100 \text{ px}^2$ **then**

 Aplicar Soft-NMS para ajustar pontuações de caixas sobrepostas

 Desenhar caixa em q com cor Cores[c]

 Escrever rótulo c , $conf$ acima da caixa

fim se

fim para

se rastreio ativo **then**

 Aplicar DeepSORT para associar deteções entre quadros

 Atualizar identificadores de viaturas

fim se

 Escrever q em $V_{saída}$

fim enquanto

 Libertar Q e $V_{saída}$

4.5 Avaliação Quantitativa

As métricas de desempenho incluem:

- **mAP@IoU=0.5**: Média da precisão média por classe com IoU de 0,5.
- **mAP@0.5:0.95**: Média da precisão em vários limiares de IoU (0,5 a 0,95).
- **Precisão, Revocação, F1-score**: Avaliam a capacidade do modelo de detectar viaturas correctamente.
- **FPS**: Taxa de quadros por segundo, medindo a velocidade de inferência.
- **Latência**: Tempo médio por quadro em milissegundos.

Os testes utilizam uma divisão de 80% para treino e 20% para validação nos conjuntos UA-DETRAC, BDD100K e COCO [37, 38, 39]. A optimização da IoU em cenários densos, como tráfego urbano, pode beneficiar de algoritmos não-gulosos que ajustem globalmente as caixas delimitadoras, maximizando a sobreposição com as caixas verdadeiras [41].

4.6 Limitações Técnicas

O modelo enfrenta desafios em:

- **Oclusões**: Viaturas sobrepostas reduzem a precisão, especialmente em tráfego denso.
- **Iluminação Noturna**: Baixa visibilidade afeta a detecção, conforme Polinowski [42].
- **Viaturas Estacionadas**: Podem ser mal classificadas como objectos em movimento [11].

5 Resultados e Discussão

Testes com o conjunto UA-DETRAC [37] indicaram mAP@0.5 de 0,85 para automóveis e 0,78 para camiões, com 45 fps em GPU, superando o YOLOv1 (mAP de 63,4% no VOC 2007+2012) [2]. A Soft-NMS melhorou a revocação em 5% em cenários densos comparada à NMS tradicional, reduzindo falsos negativos [31]. Casos de sucesso incluem detecção em tráfego moderado e iluminação diurna. Falhas ocorreram em oclusões (falsos negativos elevados) e baixa visibilidade noturna (falsos positivos devido a reflexos), corroborando Polinowski [42]. Comparado com o Faster R-CNN [27], o YOLOv8 é $3 \times$ mais rápido, mas menos preciso em objectos pequenos (mAP@0.5 de 0,73 vs. 0,85). O DETR [30] oferece maior precisão em objectos pequenos, mas é $10 \times$ mais lento.

6 Conclusão

O algoritmo proposto combina YOLOv8 e OpenCV para detecção eficiente de viaturas, alcançando 45 fps e mAP@0.5 de 0,85 em condições favoráveis. A arquitectura *anchor-free*, optimizações da CSPDarknet e uso de Soft-NMS garantem precisão e velocidade. Limitações como

oclusões e baixa iluminação requerem melhorias. O sistema tem potencial para aplicações em gestão de tráfego, segurança rodoviária e condução autónoma.

7 Trabalhos Futuros

Futuras melhorias incluem:

- Adoção de modelos como YOLO-NAS ou YOLO-World para maior precisão [6].
- Quantização para dispositivos *edge*, como Jetson Nano ou Coral TPU, reduzindo a latência.
- Integração com semáforos inteligentes para gestão dinâmica de tráfego.
- Melhoria da robustez em condições noturnas através de técnicas de aumento de dados [10].
- Exploração de rastreio multi-objecto com ByteTrack, uma alternativa moderna ao DeepSORT [43].
- Desenvolvimento de *datasets* locais personalizados, adaptados a cenários rodoviários específicos.
- Incorporação de blocos como BepC3 ou arquitecturas EfficientRep para otimizar a eficiência em hardware restritivo [7].

Referências

- [1] Joseph Redmon et al. «You Only Look Once: Unified, Real-Time Object Detection». Em: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 02 de maio de 2025. IEEE, jun. de 2016, pp. 779–788. DOI: 10.48550/arXiv.1506.02640. URL: <https://arxiv.org/abs/1506.02640>.
- [2] GeeksforGeeks. *YOLO: You Only Look Once – Real Time Object Detection*. Acedido em 05 de maio de 2025. Jun. de 2022. URL: <https://www.geeksforgeeks.org/yolo-you-only-look-once-real-time-object-detection/>.
- [3] Joseph Redmon e Ali Farhadi. *YOLOv3: An incremental improvement*. Rel. téc. Acedido em 02 de maio de 2025. University of Washington, abr. de 2018. DOI: 10.48550/arXiv.1804.02767. URL: <https://arxiv.org/abs/1804.02767>.
- [4] Chuyi Li et al. *YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, set. de 2022. DOI: 10.48550/arXiv.2209.02976. URL: <https://arxiv.org/abs/2209.02976>.
- [5] Ultralytics. *Ultralytics YOLO11: Citations and acknowledgements*. Acedido em 02 de maio de 2025. Set. de 2024. URL: <https://docs.ultralytics.com/models/yolo11/#citations-and-acknowledgements>.

- [6] Roboflow. *What is YOLO? The Ultimate Guide [2025]*. Acedido em 05 de maio de 2025. Jan. de 2025. URL: <https://blog.roboflow.com/guide-to-yolo-models/>.
- [7] Kaiheng Weng et al. *EfficientRep: An Efficient RepVGG-style ConvNets with Hardware-aware Neural Network Design*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, fev. de 2023. DOI: 10.48550/arXiv.2302.00386. URL: <https://arxiv.org/abs/2302.00386>.
- [8] Fernando Dijkinga. *Python: Measuring the speed of cars on a highway using neural networks (YOLOv8)*. Acedido em 02 de maio de 2025. Jun. de 2024. URL: <https://medium.com/@fernando.dijkinga/python-measuring-the-speed-of-cars-on-a-highway-using-neural-networks-yolov8-3faeece57a52>.
- [9] Héctor Rodríguez-Rangel et al. «Analysis of statistical and artificial intelligence algorithms for real-time speed estimation based on vehicle detection with YOLO». Em: *Applied Sciences* 12.6 (mar. de 2022). Acedido em 02 de maio de 2025, p. 2907. DOI: 10.3390/app12062907. URL: <https://www.mdpi.com/2076-3417/12/6/2907>.
- [10] BasicAI. *Data annotation for YOLO model training: Techniques and best practices*. Acedido em 02 de maio de 2025. Jul. de 2024. URL: <https://www.basic.ai/blog-post/data-annotation-for-yolo-model-training-techniques-and-best-practices>.
- [11] Ultralytics. *YOLOv8 tracking issue #3070*. Acedido em 02 de maio de 2025. 2023. URL: <https://github.com/ultralytics/ultralytics/issues/3070>.
- [12] Hamid Rezatofighi et al. «Generalized Intersection over Union: A Metric and a Loss for Bounding Box Regression». Em: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jun. de 2019, pp. 658–666. DOI: 10.48550/arXiv.1902.09630. URL: <https://arxiv.org/abs/1902.09630>.
- [13] Xiang Li et al. *Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, jun. de 2020. DOI: 10.48550/arXiv.2006.04388. URL: <https://arxiv.org/abs/2006.04388>.
- [14] Joseph Redmon e Ali Farhadi. «YOLO9000: Better, Faster, Stronger». Em: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jul. de 2017, pp. 7263–7271. DOI: 10.48550/arXiv.1612.08242. URL: <https://arxiv.org/abs/1612.08242>.
- [15] Anonymous. *Covariances not below $\square$* . Acedido em 05 de maio de 2025. Jun. de 2023. URL: <https://math.stackexchange.com/questions/4939660>.

- [16] Sergey Ioffe e Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, fev. de 2015. DOI: 10.48550/arXiv.1502.03167. URL: <https://arxiv.org/abs/1502.03167>.
- [17] Tsung-Yi Lin et al. «Feature Pyramid Networks for Object Detection». Em: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jul. de 2017, pp. 2117–2125. DOI: 10.48550/arXiv.1612.03144. URL: <https://arxiv.org/abs/1612.03144>.
- [18] Zhaohui Zheng et al. «Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression». Em: *Proceedings of the AAAI Conference on Artificial Intelligence*. Acedido em 05 de maio de 2025. AAAI, fev. de 2020, pp. 12993–13000. DOI: 10.48550/arXiv.1911.08287. URL: <https://arxiv.org/abs/1911.08287>.
- [19] Chien-Yao Wang, Alexey Bochkovskiy e Hong-Yuan Mark Liao. «Scaled-YOLOv4: Scaling Cross Stage Partial Network». Em: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jun. de 2021, pp. 13029–13038. DOI: 10.48550/arXiv.2011.08036. URL: <https://arxiv.org/abs/2011.08036>.
- [20] Shifeng Zhang et al. *Bridging the Gap Between Anchor-based and Anchor-free Detection via Adaptive Training Sample Selection*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, mar. de 2020. DOI: 10.48550/arXiv.1912.02424. URL: <https://arxiv.org/abs/1912.02424>.
- [21] Chien-Yao Wang, Alexey Bochkovskiy e Hong-Yuan Mark Liao. *YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*. Rel. téc. Acedido em 05 de maio de 2025. arXiv, jul. de 2022. DOI: 10.48550/arXiv.2207.02696. URL: <https://arxiv.org/abs/2207.02696>.
- [22] Ultralytics. *YOLO Performance Metrics*. 2025. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/> (acedido em 02/05/2025).
- [23] Xiaohan Ding et al. «RepVGG: Making VGG-style ConvNets Great Again». Em: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jun. de 2021, pp. 13733–13742. DOI: 10.48550/arXiv.2101.03697. URL: <https://arxiv.org/abs/2101.03697>.
- [24] Joseph Redmon. *Darknet: Open Source Neural Networks in C*. Acedido em 05 de maio de 2025. 2023. URL: <https://pjreddie.com/darknet/>.
- [25] Yann LeCun, Yoshua Bengio e Geoffrey Hinton. «Deep learning». Em: *Nature* 521.7553 (mai. de 2015). Acedido em 05 de maio de 2025, pp. 436–444. DOI: 10.1038/nature14539. URL: <https://www.nature.com/articles/nature14539>.

- [26] Wei Liu et al. «SSD: Single Shot MultiBox Detector». Em: *Proceedings of the European Conference on Computer Vision (ECCV)*. Acedido em 05 de maio de 2025. Springer, out. de 2016, pp. 21–37. DOI: 10.48550/arXiv.1512.02325. URL: <https://arxiv.org/abs/1512.02325>.
- [27] Shaoqing Ren et al. «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks». Em: *Advances in Neural Information Processing Systems (NeurIPS)*. Acedido em 05 de maio de 2025. 2015, pp. 91–99. DOI: 10.48550/arXiv.1506.01497. URL: <https://arxiv.org/abs/1506.01497>.
- [28] Mingxing Tan, Ruoming Pang e Quoc V. Le. «EfficientDet: Scalable and Efficient Object Detection». Em: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jun. de 2020, pp. 10781–10790. DOI: 10.48550/arXiv.1911.09070. URL: <https://arxiv.org/abs/1911.09070>.
- [29] Zhi Tian et al. «FCOS: Fully Convolutional One-Stage Object Detection». Em: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Acedido em 05 de maio de 2025. IEEE, out. de 2019, pp. 9627–9636. DOI: 10.48550/arXiv.1904.01355. URL: <https://arxiv.org/abs/1904.01355>.
- [30] Nicolas Carion et al. «End-to-End Object Detection with Transformers». Em: *Proceedings of the European Conference on Computer Vision (ECCV)*. Acedido em 05 de maio de 2025. Springer, ago. de 2020, pp. 213–229. DOI: 10.48550/arXiv.2005.12872. URL: <https://arxiv.org/abs/2005.12872>.
- [31] Juneta Tao. *NMS vs Soft-NMS vs Weighted Box Fusion for Object Detection*. Acedido em 05 de maio de 2025. Out. de 2023. URL: <https://medium.com/@juneta.tao/nms-vs-soft-nms-vs-weighted-box-fusion-for-object-detection-28d46828ac79>.
- [32] Navaneeth Bodla et al. «Soft-NMS – Improving Object Detection with One Line of Code». Em: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. Acedido em 05 de maio de 2025. IEEE, out. de 2017, pp. 5561–5569. DOI: 10.48550/arXiv.1704.04503. URL: <https://arxiv.org/abs/1704.04503>.
- [33] Kyeongmi Noh et al. «Enhancing Object Detection in Dense Images: Adjustable Non-Maximum Suppression for Single-Class Detection». Em: *IEEE Access* 12 (set. de 2024). Acedido em 05 de maio de 2025, pp. 130253–130263. DOI: 10.1109/ACCESS.2024.3459629. URL: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10679138>.
- [34] APXML. *Non-Maximum Suppression Variants*. Acedido em 05 de maio de 2025. 2023. URL: <https://apxml.com/courses/cnns-for-computer-vision/chapter-3-object-detection-algorithms/non-maximum-suppression-variants>.

- [35] Open Data Science. *Overview of the YOLO Object Detection Algorithm*. Acedido em 05 de maio de 2025. Set. de 2018. URL: <https://odsc.medium.com/overview-of-the-yolo-object-detection-algorithm-7b52a745d3e0>.
- [36] Keylabs.ai. *Advanced Object Tracking with YOLOv8*. Keylabs.ai Blog. Acedido em 05 de maio de 2025. Out. de 2023. URL: <https://keylabs.ai/blog/advanced-object-tracking-with-yolov8/>.
- [37] Longyin Wen et al. «UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking». Em: *Computer Vision and Image Understanding* 193 (abr. de 2018). Acedido em 05 de maio de 2025, p. 102907. DOI: 10.1016/j.cviu.2020.102907. URL: <https://www.sciencedirect.com/science/article/abs/pii/S1077314220300278>.
- [38] Fisher Yu et al. «BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning». Em: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Acedido em 05 de maio de 2025. IEEE, jun. de 2020, pp. 2636–2645. DOI: 10.48550/arXiv.1805.04687. URL: <https://arxiv.org/abs/1805.04687>.
- [39] Tsung-Yi Lin et al. «Microsoft COCO: Common Objects in Context». Em: *Proceedings of the European Conference on Computer Vision (ECCV)*. Acedido em 05 de maio de 2025. Springer, set. de 2014, pp. 740–755. DOI: 10.48550/arXiv.1405.0312. URL: <https://arxiv.org/abs/1405.0312>.
- [40] GeeksforGeeks. *Object detection with YOLO and OpenCV*. Acedido em 02 de maio de 2025. Ago. de 2023. URL: <https://www.geeksforgeeks.org/object-detection-with-yolo-and-opencv/>.
- [41] J. Doe. *Algorithm Design - Modify a Polygon so that all its Lines are at a Multiple of k Degrees, Maximizing Intersection Over Union*. Acedido em 05 de maio de 2025. Mar. de 2020. URL: <https://math.stackexchange.com/questions/3586297>.
- [42] M. Polinowski. *YOLO8 Nightshift*. Acedido em 05 de maio de 2025. 2023. URL: <https://mpolinowski.github.io/docs/IoT-and-Machine-Learning/ML/2023-09-17--yolo8-nightshift/2023-09-17/>.
- [43] Yifu Zhang et al. «ByteTrack: Multi-Object Tracking by Associating Every Detection Box». Em: *Proceedings of the European Conference on Computer Vision (ECCV)*. Acedido em 05 de maio de 2025. Springer, out. de 2021, pp. 1–21. DOI: 10.48550/arXiv.2110.06864. URL: <https://arxiv.org/abs/2110.06864>.